



Desarrollo de Aplicaciones Web

MATERIAL TÉCNICO DE APOYO

TAREA N° 01

Desarrollar una aplicación web aplicando los conceptos de POO en PHP.

1. FUNDAMENTOS DE PROGRAMACIÓN ORIENTADA A OBJETOS (POO) EN PHP.

La Programación Orientada a Objetos (POO) en PHP permite estructurar el código en módulos reutilizables, llamados clases, y ayuda a gestionar la complejidad de las aplicaciones web.

- **Introducción a POO en PHP.**

La POO en PHP se basa en cuatro principios fundamentales: encapsulamiento, herencia, polimorfismo y abstracción. Estos principios permiten crear software más estructurado y fácil de mantener. PHP soporta POO desde la versión 5, permitiendo definir clases y objetos que representan entidades con propiedades (atributos) y funcionalidades (métodos).

PHP también permite implementar interfaces para garantizar que las clases cumplan con un conjunto de métodos específicos. Las interfaces se definen con la palabra clave `interface` y contienen solo la declaración de métodos, sin implementación. Una clase puede implementar múltiples interfaces, lo cual permite la "herencia múltiple" a través de interfaces.

Ejemplo:

```
interface Vorable {
    public function volar();
}

interface Nadable {
    public function nadar();
}

class Pato implements Vorable, Nadable {
    public function volar() {
        echo "El pato vuela";
    }

    public function nadar() {
        echo "El pato nada";
    }
}
```

- **Clases y objetos.**

En POO, una clase es un modelo que define las propiedades y métodos que tendrán los objetos. Un objeto es una instancia de una clase, es decir, una entidad que tiene los atributos y métodos definidos en la clase.

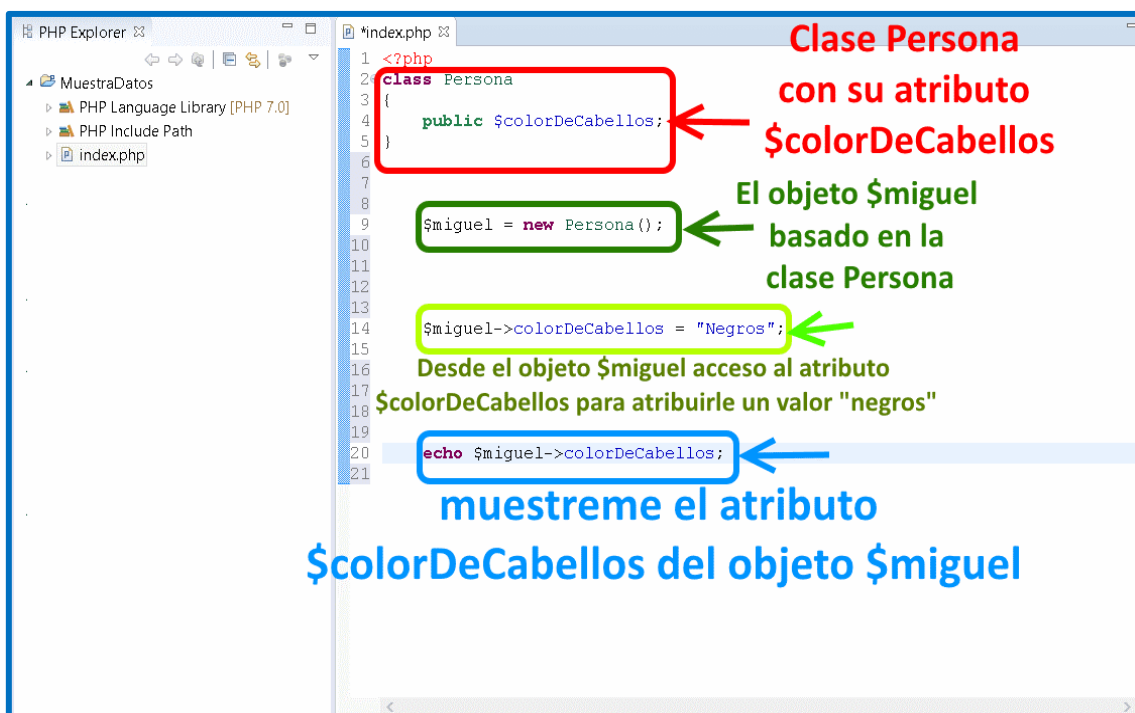
- **Declaración de una clase y creación de un objeto**

En PHP, se declara una clase usando la palabra clave class. Aquí tienes un ejemplo:

```
class Coche {
    // Atributos
    public $marca;
    public $color;

    // Método
    public function arrancar() {
        echo "El coche está arrancando";
    }
}

// Crear un objeto de la clase Coche
$coche1 = new Coche();
$coche1->marca = "Toyota";
$coche1->color = "Rojo";
$coche1->arrancar(); // Muestra: El coche está arrancando
```



Detalle de la clase Persona, con objeto Miguel y atributo colorDeCabellos.

2. ATRIBUTOS Y MÉTODOS DE UNA CLASE EN PHP.

Los atributos son variables que pertenecen a una clase, mientras que los métodos son funciones que definen el comportamiento de la clase.

- **Declaración de atributos y métodos.**

Para declarar atributos y métodos en una clase, se usan los modificadores de acceso public, protected o private, que determinan la visibilidad y accesibilidad de esos elementos.

```
class Persona {
    // Atributo público
    public $nombre;

    // Atributo privado
    private $edad;

    // Método público
    public function saludar() {
        echo "Hola, mi nombre es " . $this->nombre;
    }

    // Método para acceder a un atributo privado
    public function setEdad($edad) {
        $this->edad = $edad;
    }

    public function getEdad() {
        return $this->edad;
    }
}
```

Además de los métodos y atributos, una clase en PHP puede definir constantes con el modificador const. Las constantes son valores que no cambian y se pueden acceder sin necesidad de crear una instancia de la clase.

Ejemplo:

```
class Configuracion {
    const VERSION = "1.0";
    const AUTOR = "Desarrollador";

    public function mostrarInfo() {
        echo "Versión: " . self::VERSION . ", Autor: " . self::AUTOR;
```

```
}
}
```

```
echo Configuracion::VERSION; // Muestra: 1.0
```

PHP permite definir propiedades tipadas para especificar el tipo de dato que debe tener un atributo. Esto ayuda a mantener la consistencia de los datos en una clase y evita errores.

Ejemplo:

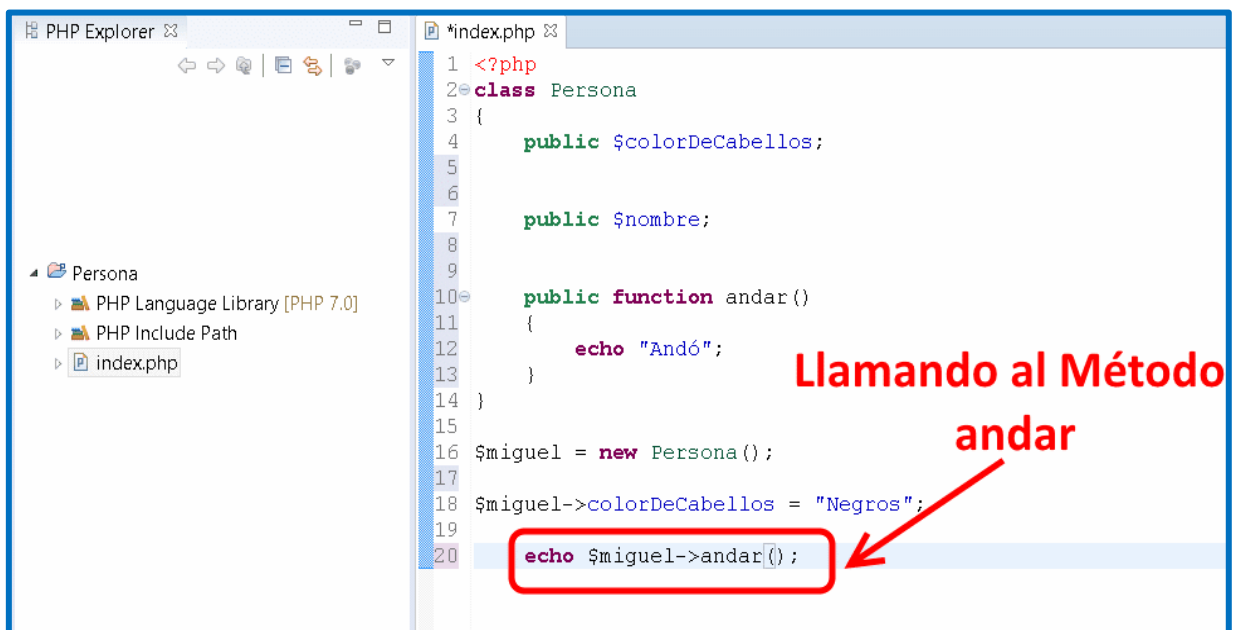
```
class Usuario {
    public string $nombre;
    private int $edad;

    public function __construct(string $nombre, int $edad) {
        $this->nombre = $nombre;
        $this->edad = $edad;
    }
}
```

- **Ejecución de métodos dentro de la clase.**

Para acceder a un método dentro de la clase, utilizamos \$this, que hace referencia al propio objeto que invoca el método.

```
$persona = new Persona();
$persona->nombre = "Juan";
$persona->setEdad(30);
$persona->saludar(); // Muestra: Hola, mi nombre es Juan
```



Llamada al método andar().

3. HERENCIA Y POLIMORFISMO EN PHP.

Herencia permite que una clase (subclase o clase hija) herede atributos y métodos de otra clase (superclase o clase padre), mientras que polimorfismo permite sobrescribir métodos en una clase hija para cambiar su comportamiento.

- **Uso de clases con herencia.**

En PHP, la herencia se establece usando la palabra clave `extends`. La clase hija puede acceder a los métodos y atributos de la clase padre (si estos son públicos o protegidos).

```
class Animal {
    public $nombre;

    public function sonido() {
        echo "El animal hace un sonido";
    }
}

class Perro extends Animal {
    public function sonido() {
        echo "El perro ladra";
    }
}

$miPerro = new Perro();
$miPerro->sonido(); // Muestra: El perro ladra
```

Herencia múltiple en PHP se puede lograr mediante el uso de traits. Un trait es un mecanismo que permite reutilizar métodos en múltiples clases. Los traits se definen con la palabra clave `trait`.

Ejemplo:

```
trait Saludable {
    public function dormir() {
        echo "Dormir 8 horas";
    }
}

trait Deportista {
    public function hacerEjercicio() {
        echo "Hacer ejercicio regularmente";
    }
}
```

```
class Persona {
    use Saludable, Deportista;
}

$persona = new Persona();
$persona->dormir(); // Muestra: Dormir 8 horas
```

- **Sobreescritura de métodos y constructor.**

- **Sobreescritura de métodos**

La sobreescritura permite redefinir un método en una clase hija, cambiando el comportamiento heredado.

```
class Gato extends Animal {
    public function sonido() {
        echo "El gato maúlla";
    }
}
```

- **Sobreescritura del constructor**

En PHP, si la clase padre tiene un constructor, la clase hija puede definir el suyo propio, o bien invocar el del padre usando `parent::__construct()`.

```
class Vehiculo {
    public $marca;

    public function __construct($marca) {
        $this->marca = $marca;
    }
}

class Moto extends Vehiculo {
    public function __construct($marca) {
        parent::__construct($marca);
        echo "Marca de la moto: " . $this->marca;
    }
}
```

```
$moto1 = new Moto("Yamaha"); // Muestra: Marca de la moto: Yamaha
```

Cuando se sobrescribe un constructor en una clase hija, también se pueden agregar parámetros adicionales o modificar el comportamiento de inicialización según las necesidades específicas de la subclase.

Ejemplo:

```
class Vehiculo {
    protected $marca;

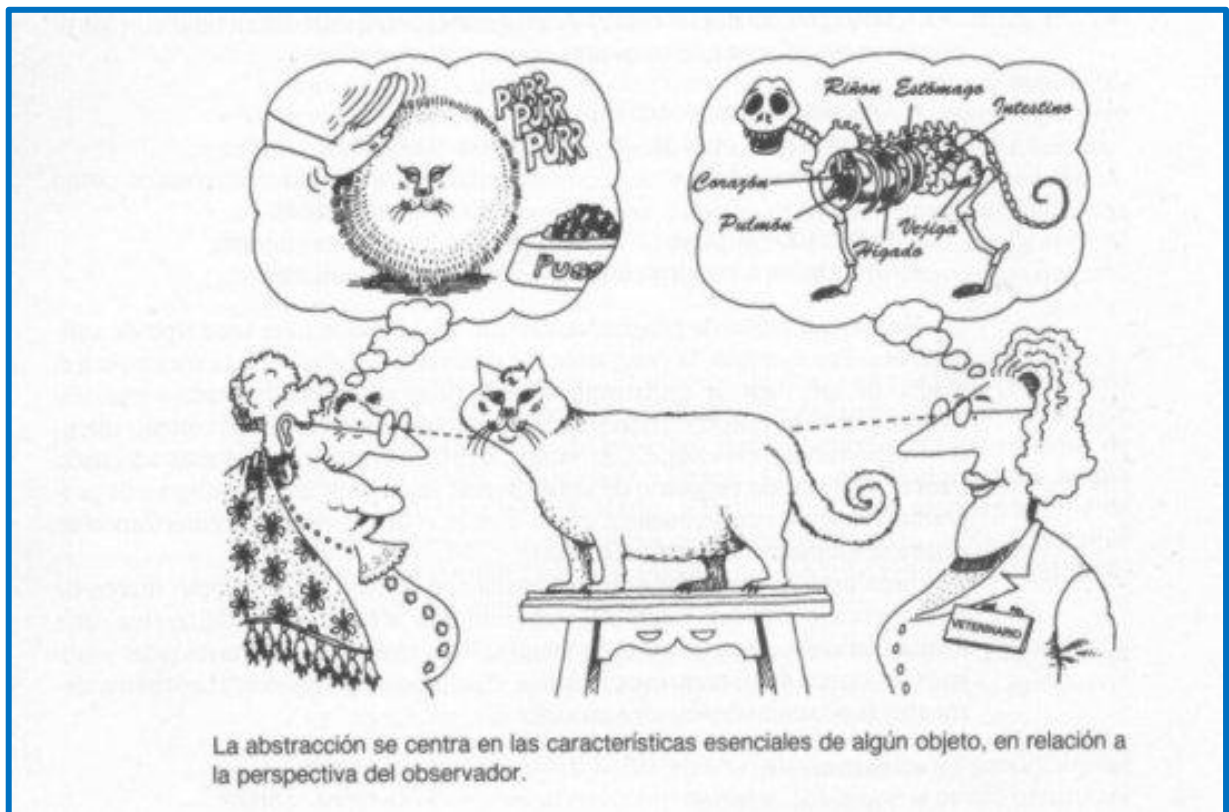
    public function __construct($marca) {
        $this->marca = $marca;
    }
}

class Carro extends Vehiculo {
    private $modelo;

    public function __construct($marca, $modelo) {
        parent::__construct($marca);
        $this->modelo = $modelo;
    }
}
```

4. ABSTRACCIÓN Y MÉTODOS ESTÁTICOS EN PHP.

La abstracción permite definir clases base que no pueden ser instanciadas, obligando a las clases hijas a implementar ciertos métodos. Los métodos estáticos son funciones de clase que se pueden llamar sin necesidad de instanciar un objeto.



[La abstracción es un proceso de interpretación y diseño](#)

- **Uso de métodos abstractos.**

Una clase abstracta se declara con `abstract`, y los métodos abstractos deben ser implementados por cualquier clase que la herede.

```
abstract class Figura {
    abstract public function area();
}

class Circulo extends Figura {
    private $radio;

    public function __construct($radio) {
        $this->radio = $radio;
    }

    public function area() {
        return pi() * pow($this->radio, 2);
    }
}

$circulo = new Circulo(5);
echo $circulo->area(); // Muestra el área del círculo
```

Una clase abstracta también puede tener métodos no abstractos. Estos métodos no abstractos pueden ser utilizados por las subclases sin necesidad de redefinirlos, lo que permite mezclar métodos comunes con métodos que requieren implementación específica.

Ejemplo:

```
abstract class Animal {
    abstract public function sonido();

    public function dormir() {
        echo "El animal está durmiendo";
    }
}

class Perro extends Animal {
    public function sonido() {
        echo "El perro ladra";
    }
}

$miPerro = new Perro();
$miPerro->dormir(); // Muestra: El animal está durmiendo
```

- **Métodos estáticos en PHP.**

Los métodos estáticos son útiles para funciones que pertenecen a la clase en sí y no a una instancia específica. Se declaran con la palabra clave `static` y se invocan usando el operador de resolución de ámbito `::`.

```
class Calculadora {  
    public static function sumar($a, $b) {  
        return $a + $b;  
    }  
}
```

```
echo Calculadora::sumar(5, 10); // Muestra: 15
```

Además de métodos estáticos, también puedes declarar propiedades estáticas en una clase. Estas propiedades pertenecen a la clase en sí y no a ninguna instancia específica. Para acceder a una propiedad estática, se utiliza el operador de resolución de ámbito `::`.

Ejemplo:

```
class Contador {  
    public static $cuenta = 0;  
  
    public static function incrementar() {  
        self::$cuenta++;  
    }  
}
```

```
Contador::incrementar();  
echo Contador::$cuenta; // Muestra: 1
```

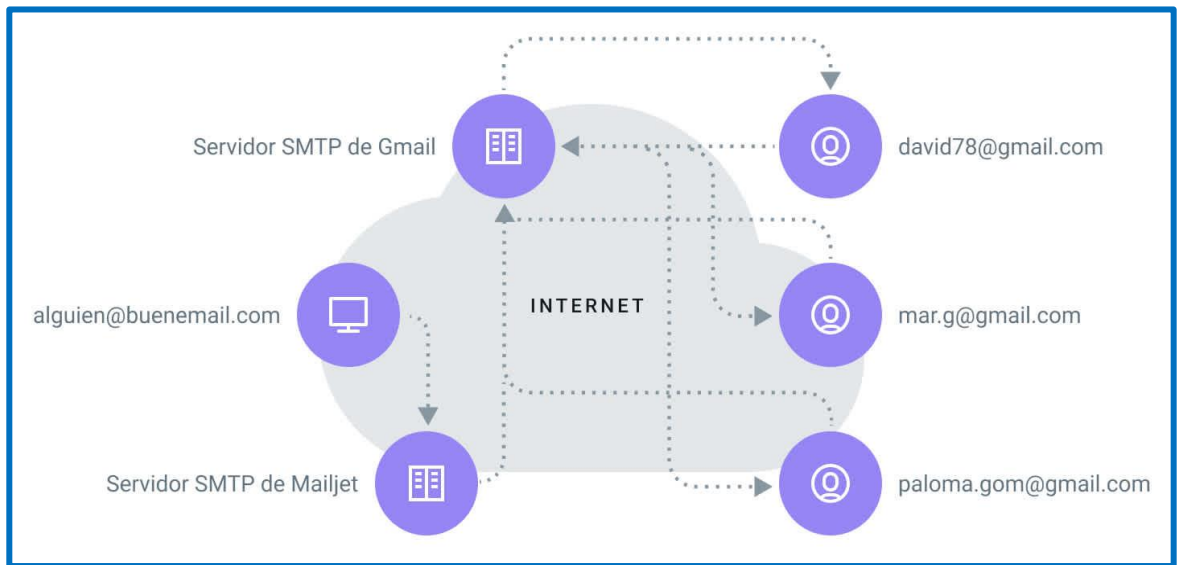
TAREA N°02

Configurar los envíos de correo electrónico de manera segura.

1. INSTALACIÓN Y CONFIGURACIÓN DE SERVICIOS DE CORREO ELECTRÓNICO.

Esta sección cubre los pasos iniciales para instalar y configurar un servidor de correo electrónico básico, además de aplicar métodos de autenticación y cifrado para mayor seguridad.

- **Instalación y configuración básica.**
 - **Instalación del servidor de correo:** Para aplicaciones que necesitan enviar correos desde el propio servidor, es común utilizar servicios como Postfix en Linux, Exim o Sendmail. En entornos Windows, se pueden usar soluciones como hMailServer.
 - **Pasos para Linux (ejemplo con Postfix):**
 - ✓ **Actualizar el sistema:** `sudo apt update`
 - ✓ **Instalar Postfix:** `sudo apt install postfix`
 - ✓ Configurar el tipo de correo que se manejará (opciones de servidor SMTP o Relay, por ejemplo).
 - **Configuración del servidor:** Después de la instalación, editar el archivo de configuración, generalmente en `/etc/postfix/main.cf`, y establecer los dominios, direcciones IP, y rutas de acceso.
 - **Configuración de Logs:** Es recomendable habilitar el registro detallado de los envíos y errores de correos para diagnóstico y seguimiento. Puedes configurar la ubicación y nivel de detalle de los logs en el servidor.
 - **Configuración de un cliente SMTP externo:** Para enviar correos de manera confiable, muchas aplicaciones usan servicios SMTP externos como SendGrid, Mailgun o SMTP de Gmail. Estos servicios ofrecen autenticación segura y tienen configuraciones específicas que deben agregarse en el código de la aplicación.



Servidor SMTP prepara y manda correos a los destinatarios.

- **Configuración de políticas de reintento y tiempo de espera:** Configurar el servidor de correo para que reintente el envío en caso de que el destinatario no responda de inmediato. En Postfix, por ejemplo, se puede configurar `bounce_queue_lifetime` y `maximal_queue_lifetime` para definir cuánto tiempo se reintenta antes de fallar definitivamente.
- **Configuración de autenticación y cifrado.**
 - **Autenticación SMTP:** Configurar el servidor SMTP para que requiera autenticación antes de permitir el envío de correos. Esto generalmente implica el uso de credenciales específicas (usuario y contraseña).
 - **Cifrado TLS/SSL:** Configurar el cifrado para proteger los datos en tránsito. TLS (Transport Layer Security) se utiliza para proteger la conexión entre el cliente y el servidor.
 - En el archivo de configuración del servidor de correo, habilitar TLS:


```
smtpd_tls_security_level = may
smtp_tls_security_level = encrypt
smtp_tls_note_starttls_offer = yes
```
 - **Verificación de certificados:** Asegurarse de que el certificado SSL utilizado para cifrar la conexión esté configurado correctamente y no haya expirado.
 - **Implementación de SPF, DKIM y DMARC:** Estas configuraciones de autenticación de correo ayudan a prevenir que los correos sean clasificados como spam o suplantados. Configurar:
 - **SPF (Sender Policy Framework):** Define qué servidores están autorizados a enviar correos en nombre del dominio.

- **DKIM (DomainKeys Identified Mail):** Agrega una firma digital a los correos que permite verificar que el mensaje no fue alterado.
- **DMARC (Domain-based Message Authentication, Reporting & Conformance):** Establece políticas de seguridad para cómo se deben manejar los correos que fallan SPF o DKIM.

2. ENVÍO DE CORREOS ELECTRÓNICOS DESDE EL ADMINISTRADOR.

Esta sección trata sobre la implementación de funcionalidades de envío de correos desde el administrador de la aplicación, como correos de notificación o confirmación.



[Correo electrónico.](#)

- **Envío de correos de notificación y confirmación.**
 - **Correos de notificación:** Se suelen enviar para informar sobre cambios en la cuenta, como actualizaciones de perfil o cambios de contraseña. Estos correos pueden ser automáticos y configurados en el código de la aplicación para ejecutarse al detectar un evento.
 - **Correos de confirmación:** Suelen utilizarse para verificar el correo electrónico del usuario en procesos de registro. El correo contiene un enlace con un token único para confirmar la dirección de correo del usuario.
 - **Programación de correos programados:** Para correos que deben enviarse periódicamente, como recordatorios o notificaciones semanales, se puede utilizar un servicio de cron en el servidor para programar estos correos de manera automatizada.

- **Configuración de una tarea en cron:** crontab -e y añadir una tarea como:

```
0 9 * * 1 /usr/bin/php /ruta/a/tu/script.php
```

- **Uso de códigos de rastreo en correos:** En correos de confirmación, incluir códigos únicos que permitan hacer seguimiento del estado del mensaje y asegurar que el usuario lo recibe.

- **Uso de funciones de PHP para envío de correos.**

- **Función mail() de PHP:** PHP tiene una función nativa mail() que permite enviar correos electrónicos. Sin embargo, esta función no es segura por sí sola y tiene limitaciones, como falta de soporte para autenticación y problemas con el spam.
- **PHPMailer y otras librerías SMTP:** PHPMailer es una biblioteca de PHP que permite un control más avanzado al enviar correos, incluyendo autenticación y cifrado.

Ejemplo básico de PHPMailer:

```
use PHPMailer\PHPMailer\PHPMailer;
require 'path/to/PHPMailer.php';
```

```
$mail = new PHPMailer();
$mail->isSMTP();
$mail->Host = 'smtp.example.com';
$mail->SMTPAuth = true;
$mail->Username = 'tu_usuario';
$mail->Password = 'tu_contraseña';
$mail->SMTPSecure = 'tls';
$mail->Port = 587;
```

```
$mail->setFrom('correo@ejemplo.com', 'Nombre');
$mail->addAddress('destinatario@ejemplo.com');
$mail->Subject = 'Asunto';
$mail->Body = 'Contenido del correo';
```

```
if($mail->send()) {
    echo "Correo enviado.";
} else {
    echo "Error: " . $mail->ErrorInfo;
}
```

- **Configuración de PHPMailer para soporte HTML y archivos adjuntos:** Enviar correos en formato HTML mejora la apariencia y permite incluir elementos visuales. También se pueden adjuntar documentos de manera segura.



[PHPMailer.](https://github.com/PHPMailer/PHPMailer)

➤ **Código para enviar correo HTML y adjuntar archivo:**

```
$mail->isHTML(true);  
$mail->Body = "<h1>Bienvenido</h1><p>Este es un correo en  
formato HTML.</p>";  
$mail->addAttachment("/ruta/al/archivo.pdf");
```

- **Configuración de SMTP Timeout:** En PHPMailer, puedes establecer el tiempo de espera para una conexión SMTP con `$mail->Timeout = 10;` para prevenir que el proceso de envío se bloquee.

3. CONSIDERACIONES DE SEGURIDAD EN ENVÍO DE CORREOS ELECTRÓNICOS.

Para proteger los datos del usuario y garantizar que los correos electrónicos se envíen de manera segura, es fundamental seguir las buenas prácticas de seguridad.

- **Lineamientos generales de seguridad.**

- **Protección contra SPAM:** Usar autenticación adecuada y configurar SPF, DKIM y DMARC en el dominio para reducir la probabilidad de que los correos sean marcados como spam.

- **Verificación de dominio:** Asegurarse de que el dominio utilizado para el envío de correos esté verificado, especialmente cuando se usa un servicio SMTP externo.
- **Uso de enlaces seguros:** Cuando se envían enlaces de confirmación o restablecimiento de contraseña, estos deben estar cifrados y contener tokens seguros que expiren después de cierto tiempo.
- **Protección contra ataques de inyección de cabeceras:** Validar y limpiar todas las entradas del usuario para evitar que se inyecten cabeceras no deseadas en los correos. En PHP, asegúrate de validar campos como nombre y asunto para evitar inyecciones:

```
$nombre = filter_var($_POST['nombre'], FILTER_SANITIZE_STRING);
$asunto = filter_var($_POST['asunto'], FILTER_SANITIZE_STRING);
```

- **Seguridad en el FileSystem para datos sensibles.**

- **Almacenamiento seguro de credenciales:** Evitar guardar credenciales de autenticación en texto plano. Utilizar archivos de configuración seguros o servicios de gestión de secretos.
- **Permisos de archivo:** Establecer permisos adecuados para archivos que contienen información sensible (por ejemplo, el archivo de configuración de PHPMailer o del cliente SMTP).
- **Cifrado de información sensible:** Utilizar cifrado para cualquier dato almacenado que se use en el envío de correos, como tokens de verificación o enlaces de restablecimiento de contraseña.
- **Almacenamiento de archivos en ubicaciones protegidas:** Evita almacenar archivos que contengan datos sensibles (como logs o configuraciones de correo) en ubicaciones accesibles desde la web, como el directorio público de la aplicación. Almacénalos en carpetas protegidas fuera del directorio raíz de acceso web.

4. INTERACCIÓN CON EL USUARIO Y MANEJO DE DATOS.

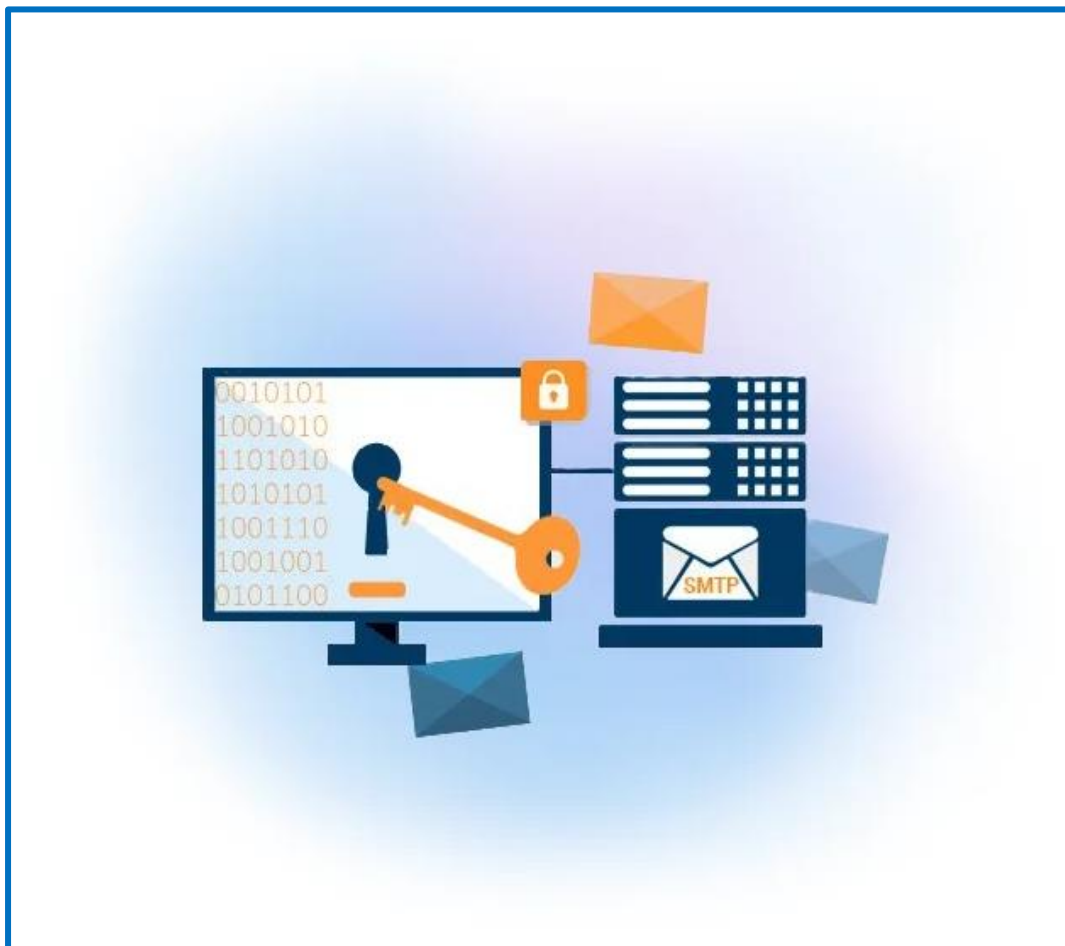
El envío de correos electrónicos debe ser una experiencia fluida para el usuario y garantizar la privacidad y seguridad de los datos proporcionados.

- **Recolección y validación de datos de usuario.**

- **Validación de correos electrónicos:** Usar validación de datos para asegurarse de que los correos electrónicos ingresados por el usuario sean válidos y estén en el formato correcto. Se puede usar la función `filter_var()` en PHP:

```
$email = "usuario@example.com";  
if (filter_var($email, FILTER_VALIDATE_EMAIL)) {  
    echo "Correo válido";  
} else {  
    echo "Correo inválido";  
}
```

- **Verificación del consentimiento del usuario:** Antes de enviar correos electrónicos, obtener el consentimiento explícito del usuario para cumplir con normativas de privacidad, como GDPR.
- **Validación de correos existentes con SMTP:** Para verificar si un correo electrónico existe, se pueden usar librerías que simulan la autenticación SMTP (sin enviar realmente un correo) para confirmar si un correo es válido. Esto requiere una configuración avanzada y debe usarse con precaución.



[Autenticación SMTP.](#)

- **Envío de correos basado en datos del usuario.**
 - **Personalización de correos electrónicos:** Adaptar el contenido del correo en función de la información proporcionada por el usuario, como su nombre o intereses.

- **Uso de plantillas de correo:** Utilizar plantillas HTML o de texto enriquecido para mejorar la presentación y profesionalismo de los correos.

Ejemplo básico de una plantilla en HTML:

```
$plantilla = file_get_contents("path/to/template.html");  
$plantilla = str_replace("{{nombre}}", $usuarioNombre, $plantilla);  
$mail->Body = $plantilla;
```

- **Pruebas y depuración de correos:** Realizar pruebas para asegurarse de que los correos llegan correctamente y que el contenido es el esperado.
- **Creación de respuestas automáticas:** Configurar respuestas automáticas que notifiquen a los usuarios que su mensaje o solicitud ha sido recibido y será atendido. Esto mejora la experiencia del usuario y proporciona tranquilidad.

Ejemplo de una respuesta automática usando PHPMailer:

```
$mail->Subject = "Gracias por contactarnos";  
$mail->Body = "Hemos recibido tu mensaje y te responderemos a la brevedad.";
```

- **Registro de envíos y auditoría:** Mantener un registro de todos los correos enviados en la base de datos, con información sobre destinatario, asunto, y fecha, para posibles auditorías o resolución de problemas.

TAREA N° 03

Elaborar páginas web PHP con AJAX.

1. USO DE CÓDIGO DE AJAX CON JAVASCRIPT.

- **Introducción a AJAX y su funcionamiento.**

AJAX (Asynchronous JavaScript and XML) es una técnica que permite actualizar partes de una página web sin recargarla completamente. AJAX utiliza JavaScript para enviar y recibir datos del servidor en segundo plano. Los datos se envían a través de solicitudes HTTP, y las respuestas pueden venir en varios formatos (JSON, XML, HTML, o texto).



[Logo de Ajax.](#)

AJAX presenta las siguientes ventajas:

- Mejora la experiencia del usuario al reducir el tiempo de carga de la página.
- Permite una interacción más dinámica.
- Reduce la carga en el servidor, ya que solo se actualizan las partes necesarias de la página.

Para comprender mejor cómo AJAX puede mejorar la eficiencia, es útil conocer algunos de los eventos de ciclo de vida de XMLHttpRequest. Los eventos principales incluyen:

- **readyState:** Representa el estado actual de la solicitud. Sus valores van desde 0 (no inicializada) hasta 4 (completada).

- **status:** Código de respuesta HTTP. Los valores comunes son 200 (éxito) y 404 (no encontrado).

Ejemplo de estados readyState en JavaScript:

```
xhr.onreadystatechange = function () {
  switch (xhr.readyState) {
    case 1:
      console.log("Solicitud establecida");
      break;
    case 2:
      console.log("Solicitud recibida por el servidor");
      break;
    case 3:
      console.log("Procesando solicitud");
      break;
    case 4:
      console.log("Solicitud completada con estado " + xhr.status);
      break;
  }
};
```

Esto ayuda a depurar y entender en qué punto puede estar fallando una solicitud en caso de errores.

- **Implementación de solicitudes AJAX con JavaScript.**

Para implementar AJAX en JavaScript, puedes utilizar el objeto XMLHttpRequest o la API fetch.

Ejemplo con XMLHttpRequest:

```
// Crear el objeto XMLHttpRequest
let xhr = new XMLHttpRequest();

// Configurar la solicitud
xhr.open("GET", "tu_archivo_php.php", true);

// Definir una función de callback
xhr.onreadystatechange = function () {
  if (xhr.readyState === 4 && xhr.status === 200) {
    document.getElementById("resultado").innerHTML =
      xhr.responseText;
  }
};
```

```
// Enviar la solicitud
xhr.send();
```

Ejemplo con fetch:

```
fetch("tu_archivo_php.php")
  .then(response => response.text())
  .then(data => {
    document.getElementById("resultado").innerHTML = data;
  })
  .catch(error => console.error("Error:", error));
```

Además de XMLHttpRequest y fetch, AJAX también puede implementarse con jQuery, una biblioteca de JavaScript que simplifica el trabajo con AJAX y permite escribir menos código. Con \$.ajax() de jQuery, puedes realizar solicitudes de manera sencilla.

Ejemplo de solicitud AJAX con jQuery:

```
$.ajax({
  url: "tu_archivo_php.php",
  type: "GET",
  success: function(data) {
    $("#resultado").html(data);
  },
  error: function(xhr, status, error) {
    console.error("Error:", error);
  }
});
```

Utilizar jQuery es ventajoso si ya está incluido en el proyecto, aunque fetch es preferible para proyectos modernos.

2. INTEGRACIÓN DE AJAX CON PHP EN EL SERVIDOR.

- **Procesamiento de solicitudes AJAX en PHP.**

En el servidor, PHP se encarga de procesar la solicitud AJAX y enviar una respuesta. Puedes acceder a los datos enviados desde JavaScript mediante \$_GET o \$_POST, dependiendo del tipo de solicitud.

Ejemplo de procesamiento en PHP:

```
// archivo: procesar_solicitud.php
if (isset($_GET['nombre'])) {
  $nombre = $_GET['nombre'];
  echo "Hola, " . htmlspecialchars($nombre) . "!";
} else {
```

```

    echo "¡No se recibió ningún nombre!";
}

```

Además de procesar las solicitudes AJAX, es importante establecer la cabecera Content-Type correcta en las respuestas para asegurarse de que el cliente interpreta los datos en el formato adecuado.

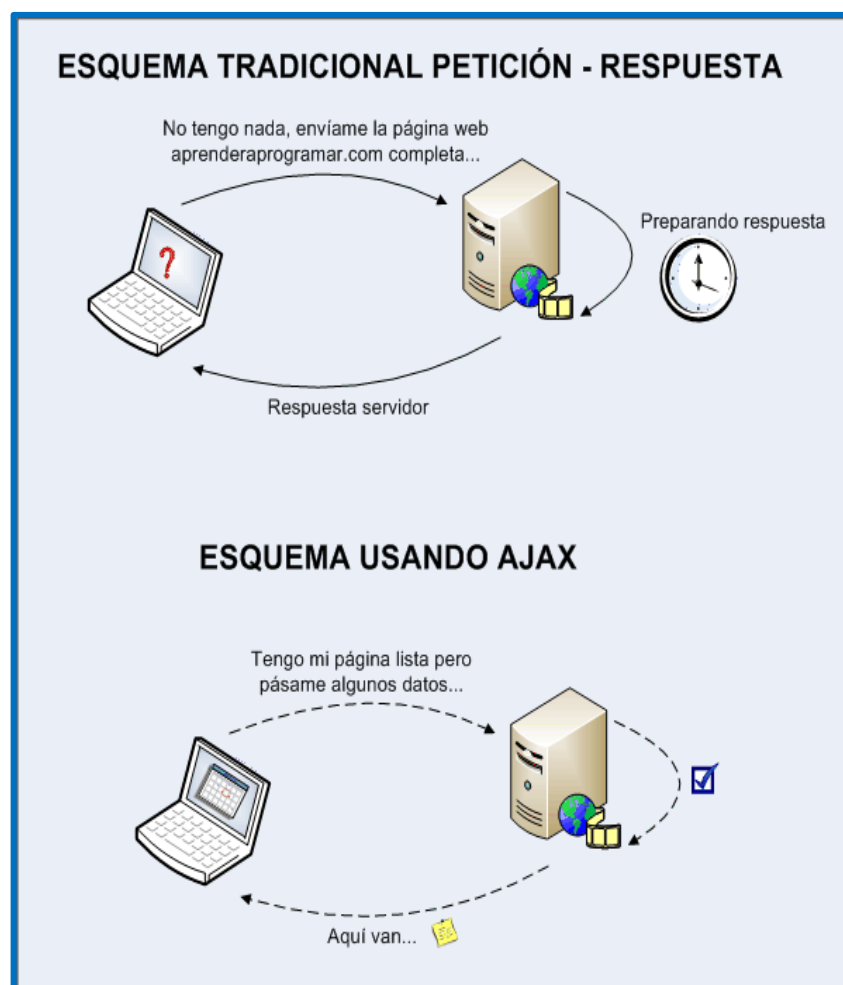
Ejemplo de cabeceras en PHP:

```

header("Content-Type: application/json");
$respuesta = array("mensaje" => "Datos recibidos correctamente");
echo json_encode($respuesta);

```

Esta práctica es esencial para aplicaciones que manejan datos en formato JSON, pues asegura que JavaScript pueda interpretar los datos correctamente.



[Comparación de esquemas.](#)

- **Manejo de respuestas y errores en AJAX.**

Es fundamental manejar tanto las respuestas exitosas como los errores para mejorar la interacción del usuario y la experiencia general.

Ejemplo en JavaScript:

```
fetch("procesar_solicitud.php?nombre=Juan")
  .then(response => {
    if (!response.ok) {
      throw new Error("Error en la solicitud");
    }
    return response.text();
  })
  .then(data => {
    document.getElementById("respuesta").innerHTML = data;
  })
  .catch(error => {
    console.error("Error:", error);
    document.getElementById("respuesta").innerHTML = "Ocurrió un
error.";
  });
```

El manejo de errores en AJAX también puede mejorarse registrando los mensajes de error y mostrando notificaciones al usuario. Una buena práctica es incluir en el servidor un código de estado HTTP apropiado cuando se detecta un error.

Ejemplo en PHP:

```
if (!isset($_GET['nombre'])) {
  http_response_code(400); // Bad Request
  echo json_encode(array("error" => "Nombre no proporcionado"));
  exit();
}
```

En JavaScript (en el cliente):

```
fetch("procesar_solicitud.php?nombre=Juan")
  .then(response => {
    if (!response.ok) {
      return response.json().then(err => { throw new Error(err.error); });
    }
    return response.text();
  })
  .then(data => {
    document.getElementById("respuesta").innerHTML = data;
  })
  .catch(error => {
    console.error("Error:", error);
    document.getElementById("respuesta").innerHTML = "Ocurrió un
error: " + error.message;
```



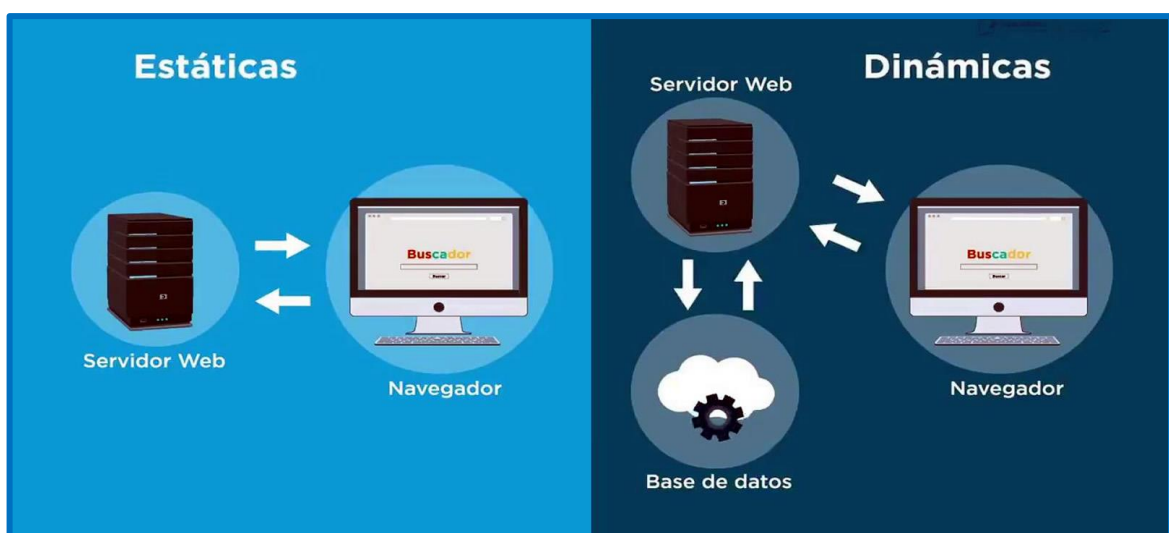
```
});
```

Esto hace que la aplicación sea más robusta y proporcione mensajes de error más específicos.

3. APLICACIONES PRÁCTICAS DE AJAX EN PÁGINAS WEB CON PHP.

- **Actualización de contenido dinámico en páginas web.**

AJAX permite que las páginas web sean interactivas y dinámicas al actualizar solo el contenido necesario, como comentarios en tiempo real, publicaciones, o cualquier información que requiera ser refrescada constantemente.



Comparación de contenido estático y dinámico.

Ejemplo de actualización de comentarios:

```
function cargarComentarios() {
    fetch("cargar_comentarios.php")
        .then(response => response.json())
        .then(data => {
            let comentariosHtml = data.map(comentario =>
                `<p>${comentario.texto}</p>`).join("");
            document.getElementById("comentarios").innerHTML =
                comentariosHtml;
        })
        .catch(error => console.error("Error:", error));
}
```

Un caso de uso práctico de AJAX es la creación de un sistema de notificaciones en tiempo real. Con AJAX, puedes configurar una consulta periódica al servidor para obtener nuevas notificaciones sin recargar la página.

Ejemplo de función de notificaciones con AJAX:

```
function obtenerNotificaciones() {
  setInterval(() => {
    fetch("obtener_notificaciones.php")
      .then(response => response.json())
      .then(data => {
        let      notificacionesHtml      =      data.map(n      =>
`<li>${n.mensaje}</li>`).join("");
        document.getElementById("notificaciones").innerHTML      =
notificacionesHtml;
      })
      .catch(error => console.error("Error:", error));
    }, 5000); // Consulta cada 5 segundos
}
```

En el servidor, obtener_notificaciones.php debería devolver un JSON con las notificaciones nuevas.

- **Implementación de AJAX para formularios interactivos.**

AJAX es útil para formularios que requieren validación en tiempo real o respuestas inmediatas sin recargar la página. Puedes usar AJAX para enviar los datos del formulario y mostrar un mensaje de éxito o error al usuario.

[Ejemplo de formulario en Ajax PHP y MySQL.](#)

Ejemplo de envío de formulario con AJAX:

```
<form id="formulario" onsubmit="enviarFormulario(event)">
  <input type="text" id="nombre" name="nombre" placeholder="Tu
nombre">
  <button type="submit">Enviar</button>
</form>
<div id="mensaje"></div>

<script>
function enviarFormulario(event) {
  event.preventDefault(); // Evitar recargar la página
  let nombre = document.getElementById("nombre").value;

  fetch("procesar_formulario.php", {
    method: "POST",
    headers: { "Content-Type": "application/x-www-form-urlencoded" },
    body: `nombre=${nombre}`
  })
  .then(response => response.text())
  .then(data => {
    document.getElementById("mensaje").innerHTML = data;
  })
  .catch(error => console.error("Error:", error));
}
</script>
```

En PHP (procesar_formulario.php):

```
if (isset($_POST['nombre'])) {
  $nombre = $_POST['nombre'];
  echo "Gracias por tu mensaje, " . htmlspecialchars($nombre) . "!";
} else {
  echo "Por favor, ingresa tu nombre.";
}
```

Para mejorar aún más la experiencia del usuario, AJAX permite mostrar mensajes de error en tiempo real y proporcionar feedback inmediato para cada campo de entrada del formulario.

Ejemplo de validación en tiempo real de un formulario:

```
document.getElementById("nombre").addEventListener("input", function() {
  let nombre = this.value;
  fetch("validar_nombre.php?nombre=" + nombre)
```

```

.then(response => response.json())
.then(data => {
    if (data.valido) {
        document.getElementById("mensaje").innerHTML = "Nombre
válido";
        document.getElementById("mensaje").style.color = "green";
    } else {
        document.getElementById("mensaje").innerHTML = "Nombre no
disponible";
        document.getElementById("mensaje").style.color = "red";
    }
})
.catch(error => console.error("Error:", error));
});

```

En el archivo validar_nombre.php, puedes validar el nombre recibido y devolver un JSON indicando si es válido o no.

Ejemplo en PHP (validar_nombre.php):

```

$nombre = $_GET['nombre'] ?? "";
$nombreValido = ($nombre != 'Juan'); // Por ejemplo, Juan ya existe
echo json_encode(array("valido" => $nombreValido));

```

Esta técnica es útil para validar campos como nombres de usuario, correos electrónicos o contraseñas antes de que el usuario envíe el formulario completo.

TAREA N° 04

Desarrollar una aplicación web en PHP con Patrones de Diseño.

1. CONCEPTO DE PATRONES DE DISEÑO EN DESARROLLO WEB.

Los patrones de diseño en desarrollo web también ayudan a mejorar la colaboración entre desarrolladores. Al seguir estructuras conocidas, es más fácil entender y modificar el código, permitiendo una colaboración más fluida en proyectos grandes. Estos patrones no solo son específicos de un lenguaje, sino que sus principios pueden aplicarse en cualquier entorno de programación.



[Patrones de diseño.](#)

Los patrones de diseño son soluciones probadas para problemas comunes de diseño de software. Son estructuras reutilizables que ayudan a organizar el código de una manera mantenible, facilitando la legibilidad y optimización de las aplicaciones.

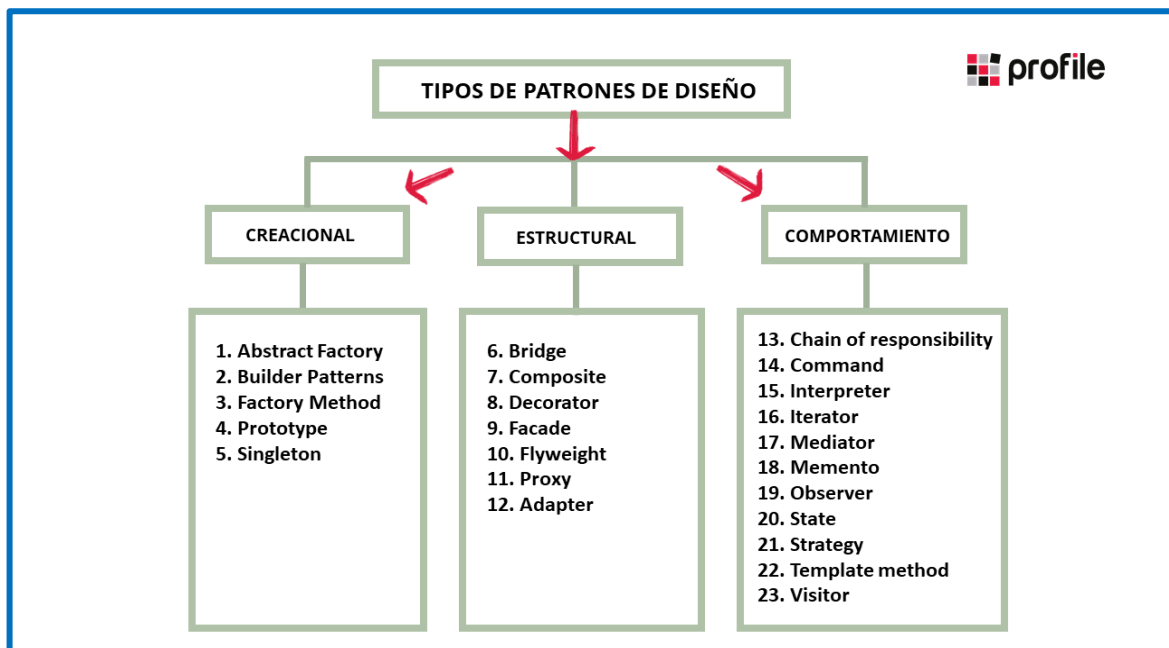
- **Definición y objetivo de los patrones de diseño.**

- **Definición:** Los patrones de diseño son prácticas y soluciones reutilizables que ayudan a resolver problemas específicos de diseño de software, proporcionando una estructura que facilita la implementación de funcionalidades de manera eficiente.
- **Objetivo:** Ayudar a los desarrolladores a evitar la reinención de soluciones para problemas comunes. Facilitan la creación de sistemas modulares, escalables y mantenibles, y promueven una arquitectura de software de alta calidad.

- **Clasificación de los patrones de diseño.**

Los patrones de diseño se dividen en tres categorías principales:

- **Patrones creacionales:** Enfocados en la creación de objetos de manera controlada y flexible.
- **Patrones estructurales:** Ayudan a organizar y estructurar las clases y objetos.
- **Patrones de comportamiento:** Se centran en la comunicación y la responsabilidad entre los objetos.



[Tipos de patrones de diseño.](#)

La elección del tipo de patrón depende de la naturaleza del problema. Por ejemplo, los patrones creacionales son ideales para aplicaciones que requieren una administración específica en la creación de objetos, mientras que los estructurales son útiles para organizar elementos visuales y estructurales en la interfaz de usuario.

A continuación, veremos cada uno de ellos:

- **Patrones creacionales**

Estos patrones permiten instanciar objetos en distintos contextos y controlar su ciclo de vida. Ejemplos en desarrollo web incluyen el patrón Singleton (para instancias únicas) y Factory (para instancias basadas en tipos o parámetros específicos).

Otros patrones creacionales incluyen Prototype, que permite crear copias de objetos sin depender de sus clases concretas. Este patrón es útil en sistemas con muchos objetos que tienen características similares pero que necesitan ser modificados de manera individual.

- **Patrones estructurales**

Los patrones estructurales se utilizan para crear relaciones entre objetos y clases, formando estructuras flexibles y eficientes. Ejemplos incluyen el patrón Decorador, que añade funcionalidades a un objeto sin alterar su estructura base.

En el desarrollo web, el patrón Adapter también es popular, ya que permite que interfaces de clases incompatibles trabajen juntas. Esto es útil cuando se integran servicios de terceros que tienen una API diferente a la que utiliza el sistema en desarrollo.

- **Patrones de comportamiento**

Estos patrones facilitan la comunicación y la asignación de responsabilidades entre objetos. En desarrollo web, son útiles para gestionar interacciones, lógica y comportamientos compartidos, mejorando la respuesta y adaptabilidad de la aplicación.

El patrón Observer es un ejemplo clásico en aplicaciones web. Se utiliza para permitir que varios objetos se suscriban a cambios en otro objeto. En desarrollo web, esto es útil para actualizar múltiples partes de la interfaz cuando el estado de una variable o recurso cambia, como en una aplicación en tiempo real.

2. APLICACIÓN DE DIVERSOS PATRONES DE DISEÑO WEB EN PHP.

En PHP, los patrones de diseño permiten una implementación más organizada y estructurada del código. Estos patrones no solo mejoran el rendimiento, sino que también ayudan a reducir la duplicación de código y facilitan futuras modificaciones.

- **Implementación de patrones en PHP.**

PHP es un lenguaje orientado a objetos, por lo que permite la implementación de la mayoría de los patrones de diseño. Usando clases y objetos, se pueden aplicar patrones que optimizan el uso de recursos y mejoran la escalabilidad.

- **Patrones comunes en desarrollo web.**

Aplicar patrones de diseño en PHP no solo optimiza la estructura del código, sino que también mejora la seguridad y la escalabilidad de las aplicaciones. Por ejemplo, el uso adecuado de patrones puede ayudar a proteger datos sensibles y simplificar la adición de nuevas características a la aplicación.

A continuación, se presentan algunos patrones comunes en PHP que se aplican a aplicaciones web.

- **Patrón Singleton**

El patrón Singleton asegura que una clase tenga una única instancia y proporciona un punto de acceso global a esa instancia. Es útil para manejar conexiones a bases de datos u objetos de configuración.

Ejemplo de Singleton en PHP:

```
class Database {
    private static $instance = null;
    private function __construct() {}
    public static function getInstance() {
        if (self::$instance === null) {
            self::$instance = new Database();
        }
        return self::$instance;
    }
}
```

Una desventaja potencial del patrón Singleton es que puede dificultar la realización de pruebas unitarias, ya que su naturaleza de instancia única puede interferir en los entornos de prueba. Por eso, es importante considerar alternativas o excepciones al usar este patrón en aplicaciones con alta necesidad de pruebas automatizadas.

- **Patrón Factory**

El patrón Factory es un patrón creacional que permite crear objetos sin especificar su clase exacta. En su lugar, define una interfaz para la

creación de objetos, delegando la instancia a subclases.

Ejemplo de Factory en PHP:

```
interface Product {
    public function getType();
}

class ProductA implements Product {
    public function getType() {
        return "ProductA";
    }
}

class ProductFactory {
    public static function createProduct($type) {
        if ($type === "A") {
            return new ProductA();
        }
        // Añadir más tipos según sea necesario
    }
}
```

El patrón Factory también es útil para la gestión de dependencias, ya que permite desacoplar la creación de objetos de su implementación. Esto es útil en aplicaciones web que necesitan instancias de objetos específicos para diferentes configuraciones o ambientes.

○ Patrón Decorador

El patrón Decorador permite añadir funcionalidad adicional a un objeto de forma dinámica. Es útil para extender las capacidades de un objeto sin modificar su código original.

Ejemplo de Decorador en PHP:

```
interface Notifier {
    public function send($message);
}

class BasicNotifier implements Notifier {
    public function send($message) {
        echo "Mensaje: $message";
    }
}

class EmailNotifierDecorator implements Notifier {
```

```

private $notifier;
public function __construct(Notifier $notifier) {
    $this->notifier = $notifier;
}
public function send($message) {
    $this->notifier->send($message);
    echo " Notificación enviada por correo.";
}
}

```

```

$notifier = new EmailNotifierDecorator(new BasicNotifier());
$notifier->send("Nuevo mensaje");

```

El patrón Decorador es especialmente útil para aplicaciones de comercio electrónico, donde se pueden agregar opciones adicionales, como impuestos, envíos o descuentos, a un producto base sin modificar su clase original.

3. APLICACIÓN DEL MODELO VISTA CONTROLADOR (MVC) EN UN CRUD.

El patrón MVC es uno de los patrones estructurales más populares para desarrollar aplicaciones web. Permite separar de manera clara la lógica de negocio, la presentación y el control de flujo de la aplicación, mejorando la mantenibilidad y escalabilidad.



[Modelo vista controlador \(MVC\)](#)

- **Estructura del modelo-vista-controlador.**
 - **Modelo:** Interactúa con la base de datos, define la estructura y operaciones de los datos.

- **Vista:** Presenta la interfaz de usuario y muestra los datos.
- **Controlador:** Recibe y gestiona las solicitudes, actuando como intermediario entre el modelo y la vista.
- **Implementación de un CRUD usando MVC en PHP.**

A continuación, un ejemplo simplificado para un CRUD de una tabla "usuarios".

- **Estructura básica:**

- **Modelo (User.php):** Define la clase User para interactuar con la tabla.
- **Vista (views/user_list.php):** Muestra los usuarios en una lista.
- **Controlador (UserController.php):** Procesa las solicitudes y muestra la vista correspondiente.

- **Código de ejemplo:**

- **Modelo (User.php):**

```
class User {
    private $db;
    public function __construct($db) {
        $this->db = $db;
    }
    public function getAllUsers() {
        return $this->db->query("SELECT * FROM usuarios")-
        >fetchAll();
    }
    // Métodos para crear, actualizar, y eliminar usuarios
}
```

- **Controlador (UserController.php):**

```
require_once 'User.php';
class UserController {
    private $model;
    public function __construct($db) {
        $this->model = new User($db);
    }
    public function listUsers() {
        $users = $this->model->getAllUsers();
        include 'views/user_list.php';
    }
}
```

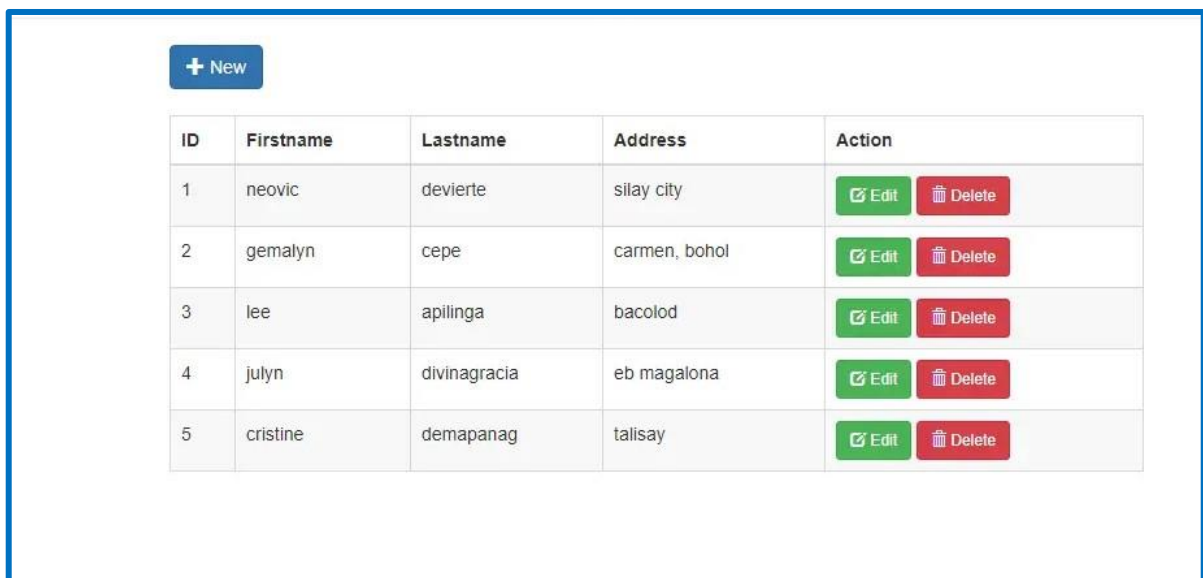
```
// Métodos para procesar creación, actualización y eliminación
}
```

➤ **Vista (views/user_list.php):**

```
foreach ($users as $user) {
    echo "<p>{$user['nombre']}</p>";
}
```

○ **Explicación del flujo CRUD:**

- **Crear:** El controlador recibe la solicitud de crear un nuevo usuario y usa el modelo para añadirlo a la base de datos.
- **Leer:** El controlador consulta los usuarios y envía los datos a la vista.
- **Actualizar:** El controlador recibe datos modificados, los envía al modelo para actualizar en la base de datos y refresca la vista.
- **Eliminar:** El controlador solicita al modelo que elimine un usuario y actualiza la vista para reflejar los cambios.



ID	Firstname	Lastname	Address	Action
1	neovic	devierte	silay city	 Edit  Delete
2	gemalyn	cepe	carmen, bohol	 Edit  Delete
3	lee	apilinga	bacolod	 Edit  Delete
4	julyn	divinagracia	eb magalona	 Edit  Delete
5	cristine	demapanag	talisay	 Edit  Delete

[Imagen referencial de la implementación de un CRUD en MVC.](#)

Una práctica recomendada en la implementación de un CRUD en MVC es utilizar la inyección de dependencias para el manejo de bases de datos. Esto permite que el controlador utilice una instancia específica de conexión a la base de datos, facilitando la personalización de los controladores para ambientes de desarrollo y producción.

TAREA N° 05

Desarrollar una aplicación web en PHP con Servicios web en PHP.

1. CONCEPTO DE SERVICIOS WEB.

Los servicios web permiten la comunicación entre aplicaciones a través de la web, usando estándares de red abiertos. Esto facilita que sistemas y lenguajes distintos puedan interoperar, permitiendo intercambiar datos de manera flexible. Los servicios web facilitan el principio de interoperabilidad entre aplicaciones, lo que permite que sistemas heterogéneos (diferentes lenguajes, plataformas o ubicaciones geográficas) se comuniquen de manera efectiva. Este modelo sigue la filosofía SOA (Arquitectura Orientada a Servicios), que prioriza la modularidad y la reutilización.

- **Definición de servicios web.**

Un servicio web es una aplicación o componente accesible a través de una red (normalmente la web) que permite la interacción entre sistemas mediante protocolos estándar como HTTP. Su objetivo es facilitar la comunicación y el intercambio de datos, permitiendo que diferentes aplicaciones y plataformas se conecten entre sí.

- **Tipos de servicios web.**

Los servicios web se clasifican en función de sus protocolos y arquitecturas. Los dos tipos más comunes son SOAP y REST.

- **SOAP (Simple Object Access Protocol)**

SOAP es un protocolo de mensajería basado en XML que permite la comunicación entre aplicaciones. Está orientado a contratos y requiere una estructura estricta en el formato de sus mensajes, lo que lo hace robusto, pero también más complejo. SOAP suele usarse en aplicaciones empresariales donde la seguridad, la transacción y la confiabilidad son esenciales.

Las ventajas de SOAP son:

- **Seguridad avanzada:** Integra WS-Security para asegurar los mensajes.
- **Transacciones complejas:** Ideal para sistemas que requieren operaciones complejas.

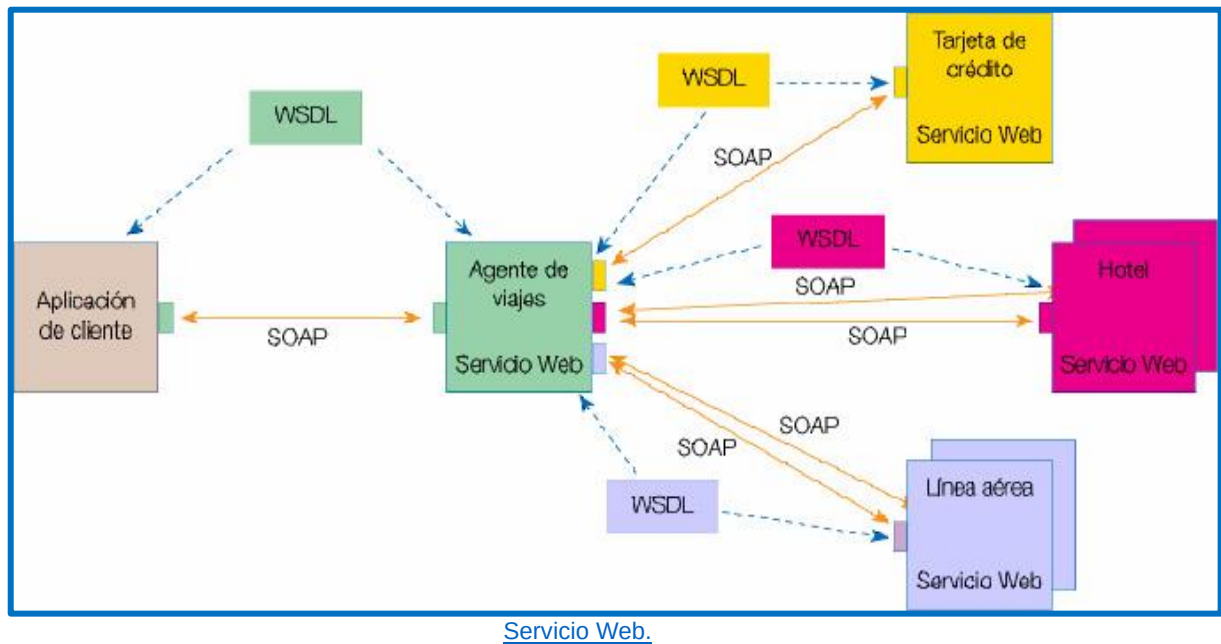
- **Soporte de servicios críticos:** Proporciona alta confiabilidad.

SOAP también es compatible con varios protocolos de transporte, como HTTP, SMTP, y FTP, lo cual ofrece flexibilidad adicional en entornos empresariales donde el tráfico puede requerir diferentes protocolos según la red y las necesidades de la organización.

Las desventajas de SOAP son:

- Complejidad en la implementación.
- Sobrecarga de ancho de banda debido a su estructura XML detallada.

A pesar de sus limitaciones en términos de rendimiento, SOAP sigue siendo una opción preferida en aplicaciones de servicios críticos, como banca y atención médica, donde la confiabilidad y la estandarización son factores clave.



○ **REST (Representational State Transfer)**

REST es una arquitectura de servicios web que usa HTTP y URL para el intercambio de datos. Su enfoque es menos formal que SOAP, centrándose en la simplicidad y en el uso de métodos HTTP como GET, POST, PUT y DELETE. REST se ha convertido en el estándar para aplicaciones web debido a su facilidad de uso y eficiencia.

Las ventajas de REST son:

- **Ligereza:** Menos consumo de ancho de banda en comparación con SOAP.

- **Escalabilidad:** Más adecuado para aplicaciones web móviles y sistemas de microservicios.
- **Facilidad de uso:** Simple y fácil de entender.

Las desventajas de REST son:

- **Seguridad limitada:** Sin soporte integrado para WS-Security.
- **Sin soporte de transacciones:** No es ideal para operaciones que requieran alta consistencia.

2. UTILIDAD Y APLICACIONES DE UN SERVICIO WEB.

Los servicios web son fundamentales en el desarrollo de aplicaciones modernas, permitiendo la conexión entre distintas aplicaciones y sistemas. Su uso abarca desde simples consultas de datos hasta complejas transacciones entre múltiples sistemas.

- **Casos de uso de servicios web en aplicaciones web.**

En el desarrollo moderno, los servicios web también son fundamentales en el ecosistema de microservicios. Este enfoque permite crear aplicaciones divididas en servicios independientes que se comunican entre sí mediante APIs, facilitando la escalabilidad horizontal y mejorando el tiempo de respuesta y la eficiencia del sistema.

- **Autenticación de usuario:** APIs que permiten la verificación de identidad mediante servicios externos, como OAuth.
- **Procesamiento de pagos:** Servicios de pago como PayPal o Stripe ofrecen APIs para integrarse fácilmente en aplicaciones web.
- **Intercambio de datos en tiempo real:** Servicios de terceros que permiten el acceso a datos en tiempo real, como clima o precios de mercado.
- **Aplicaciones de mensajería y notificaciones:** APIs de servicios como Twilio permiten el envío de SMS y notificaciones automáticas.

- **Beneficios de los servicios web para la escalabilidad.**

- **Descentralización:** Los servicios web permiten dividir la aplicación en componentes independientes, lo que facilita la escalabilidad.
- **Reutilización de componentes:** Con APIs bien definidas, otros desarrolladores pueden usar y extender funcionalidades existentes sin modificar el código base.

- **Reducción de costos:** Al facilitar la integración de servicios de terceros, las aplicaciones pueden crecer sin necesidad de desarrollar todas las funciones desde cero.

3. USO DE LIBRERÍAS PARA INTERACTUAR CON ARCHIVOS Y SERVICIOS EXTERNOS.

Las librerías en PHP ayudan a manejar y procesar archivos y facilitan la integración con servicios externos, lo cual es crucial para implementar servicios web.

- **Librerías para manejo de archivos en PHP.**

PHP proporciona diversas librerías para manipular archivos, tanto para leer, escribir, y almacenar datos:

- **File System Functions:** PHP incluye funciones nativas como `fopen()`, `fwrite()`, `fread()`, y `file_get_contents()` para leer y escribir archivos.
- **FilesystemIterator:** Útil para la gestión de directorios completos.
- **SplFileObject:** Proporciona un manejo avanzado de archivos, permitiendo una manipulación orientada a objetos de los mismos.

Ejemplo:

```
$file = new SplFileObject("archivo.txt", "w+");  
$file->fwrite("Contenido del archivo");
```

- **Uso de librerías externas y APIs.**

- **cURL:** Utilizado para realizar solicitudes HTTP a servicios web, muy útil para interactuar con APIs.



[Logo de curl.](#)

- **Guzzle:** Es una librería de PHP que simplifica las solicitudes HTTP. Es preferida en proyectos modernos, ya que permite manejar peticiones y respuestas de forma más estructurada.



Logo de Guzzle.

Ejemplo:

```
$ch = curl_init("https://api.example.com/data");
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
$response = curl_exec($ch);
curl_close($ch);
```

Otra herramienta popular es JWT (JSON Web Token), una solución para manejar la autenticación y autorización de usuarios en servicios web. JWT permite la transferencia segura de información entre aplicaciones, manteniendo la integridad de los datos a través de la firma digital.

4. DISPOSICIÓN Y PUBLICACIÓN DE SERVICIOS WEB EN PHP.

La publicación de servicios web en PHP permite que otros sistemas accedan a los recursos y funcionalidades de una aplicación.

- **Creación de un servicio web en PHP.**

Para crear un servicio web en PHP, se utiliza un archivo que responde a solicitudes HTTP, comúnmente en formato JSON. A continuación, se muestra cómo crear un servicio básico de API en PHP.

Ejemplo:

```
header("Content-Type: application/json");

if ($_SERVER['REQUEST_METHOD'] == 'GET') {
    $data = ["mensaje" => "Bienvenido al servicio web en PHP"];
    echo json_encode($data);
}
```

- **Publicación de un servicio Web en un servidor Web.**

Para publicar un servicio web, es necesario configurar el entorno de servidor, como Apache o Nginx, de manera adecuada y gestionar aspectos de seguridad. A continuación, algunos pasos para la publicación:

- **Configurar el servidor:** Asegúrate de que el servidor tenga soporte para PHP y que esté configurado para responder a solicitudes HTTP.
- **Configurar CORS (Cross-Origin Resource Sharing):** Si el servicio será accedido desde diferentes dominios, debes habilitar el acceso CORS en el servidor.
- **Seguridad y autenticación:** Implementar medidas de seguridad como tokens de autenticación para limitar el acceso no autorizado.



[Servicio web en PHP.](#)

Además de la configuración básica, es importante considerar el uso de servicios de monitoreo para medir el rendimiento y la disponibilidad del servicio web en producción. Herramientas como New Relic o DataDog ofrecen monitoreo en tiempo real, permitiendo detectar y resolver problemas de rendimiento rápidamente.

TAREA N° 06

Configurar un entorno web en PHP con Framework.

1. DEFINICIÓN DE FRAMEWORK EN DESARROLLO WEB.

- **Concepto de Framework.**

Los frameworks son herramientas que facilitan y aceleran el desarrollo de aplicaciones al ofrecer una estructura predefinida. En el contexto de PHP, un framework proporciona un conjunto de componentes, bibliotecas, y patrones de diseño que ayudan a desarrollar aplicaciones de manera más eficiente, escalable y segura. Los frameworks PHP siguen normalmente el patrón Modelo-Vista-Controlador (MVC), que separa la lógica de negocio, la interfaz de usuario y el control de flujo en módulos independientes, mejorando la organización y el mantenimiento del código.

Además, los frameworks suelen proporcionar herramientas de línea de comandos que facilitan tareas comunes, como la creación de modelos, controladores y migraciones de bases de datos. Por ejemplo, en Laravel, el comando `php artisan` permite acceder a una serie de herramientas que automatizan y simplifican el desarrollo, mejorando la productividad.

- **Tipos de Frameworks en PHP.**

Existen varios tipos de frameworks en PHP, y cada uno está diseñado para diferentes propósitos y niveles de complejidad:

- **Frameworks de propósito general:** Estos ofrecen un conjunto amplio de herramientas y funcionalidades para crear aplicaciones complejas. Ejemplos: Laravel, Symfony.
- **Micro-frameworks:** Ligeros y diseñados para proyectos más pequeños que no necesitan toda la funcionalidad de un framework completo. Ejemplo: Slim, Lumen.
- **Frameworks orientados a API:** Enfocados en facilitar el desarrollo de APIs RESTful, ideales para aplicaciones backend. Ejemplo: Lumen.
- **Frameworks de comercio electrónico:** Especializados en el desarrollo de tiendas en línea y aplicaciones de comercio electrónico. Estos frameworks suelen integrar funcionalidades para gestionar productos, clientes y pagos. Ejemplos: Magento, OpenCart.
- **Frameworks con enfoque en rendimiento:** Diseñados para alta velocidad y bajo consumo de recursos. Ejemplo: Phalcon.

2. USO DE LIBRERÍAS EXTERNAS EN FRAMEWORKS PHP.

- **Incorporación de librerías y componentes.**

Los frameworks PHP suelen soportar la incorporación de librerías externas mediante herramientas como Composer, que es el gestor de dependencias estándar en PHP. Composer permite instalar, actualizar y mantener las librerías de manera organizada, integrándolas directamente en el proyecto.



[Logo de Composer.](#)

Ejemplo de cómo añadir una librería con Composer:

```
composer require guzzlehttp/guzzle
```

Este comando instalará Guzzle, una librería PHP para realizar solicitudes HTTP, y la integrará con el autoloader de Composer, facilitando su uso dentro de cualquier framework PHP.

Algunas librerías externas populares que se integran comúnmente en frameworks PHP incluyen Carbon para manejo avanzado de fechas y tiempos, PHPUnit para pruebas automatizadas, y SwiftMailer para enviar correos electrónicos. Estas herramientas agregan funcionalidades específicas, reduciendo el tiempo de desarrollo al ofrecer soluciones listas para usar.

- **Buenas prácticas para integrar librerías.**

- **Compatibilidad de versiones:** Asegúrate de que las versiones de las librerías sean compatibles con la versión de PHP y con otros componentes del framework.

- **Optimización del autoload:** Usa el comando `composer dump-autoload -o` para optimizar el archivo de autoload y mejorar el rendimiento.
- **Uso de interfaces:** Al integrar una librería externa, es útil utilizar interfaces que actúen como intermediarios, lo que permitirá cambiar la librería en el futuro sin afectar todo el código.
- **Validación y seguridad:** Verifica la confiabilidad de las librerías y mantén las dependencias actualizadas para evitar vulnerabilidades.

Es también recomendable usar `composer.lock` para mantener versiones específicas de las librerías en todos los entornos, evitando incompatibilidades al asegurar que todos los desarrolladores del equipo trabajen con las mismas dependencias. Para esto, puedes usar el comando `composer install`, que mantiene las versiones bloqueadas según lo indicado en `composer.lock`.

3. INSTALACIÓN Y CONFIGURACIÓN DE UN FRAMEWORK PHP.

- **Instalación de un framework en un entorno de desarrollo.**

La instalación de un framework PHP varía según el framework que elijas, pero generalmente se realiza con Composer. A continuación, se describe cómo instalar Laravel como ejemplo:

- **Requisitos previos:** Asegúrate de que PHP, Composer y un servidor web (como Apache o Nginx) estén correctamente configurados en tu entorno.



[Logo de Apache.](#)



[Logo de Nginx.](#)

- **Instalación con Composer:**

```
composer create-project --prefer-dist laravel/laravel nombre-proyecto
```

Esto crea una nueva aplicación Laravel en el directorio especificado.

- **Configuración del entorno:** Al instalar, Laravel generará un archivo .env que contiene configuraciones básicas como la conexión a la base de datos, el nombre de la aplicación y las credenciales de autenticación.



[Logo de Laravel.](#)

Al iniciar el proyecto, es recomendable utilizar un entorno virtualizado o contenedor, como Docker, para garantizar la portabilidad y consistencia del entorno de desarrollo. Docker permite crear una configuración de servidor específica para el framework PHP seleccionado, integrando la base de datos, el servidor web y PHP en un solo contenedor que puede replicarse fácilmente en otros entornos.

- **Configuración del Framework para el desarrollo web.**

Cada framework requiere una configuración inicial específica para adaptarse al entorno de desarrollo:

- **Configuración del entorno:** Personaliza el archivo `.env` con información específica del entorno (local, de prueba o de producción). Por ejemplo, define el nombre de la base de datos y las credenciales en el archivo `.env` en Laravel:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=nombre_base_datos
DB_USERNAME=usuario
DB_PASSWORD=contraseña
```

- **Configuración de URLs y rutas:** Establece la URL base para el proyecto y verifica que el servidor web (Apache o Nginx) esté apuntando a la carpeta pública del framework (por ejemplo, la carpeta `public` en Laravel).

Para mejorar el rendimiento en producción, muchos frameworks permiten habilitar un sistema de cacheo, que guarda consultas y datos de uso frecuente en memoria para reducir el tiempo de carga de la aplicación. En Laravel, por ejemplo, se pueden utilizar cachés de vistas, rutas y consultas de base de datos. Puedes activar el caché de rutas usando el comando `php artisan route:cache`, lo cual ayuda a optimizar la carga en entornos de producción.

- **Habilitación de características de desarrollo:** Activa opciones como el modo debug para mostrar errores detallados durante el desarrollo. En Laravel, esto se hace configurando `APP_DEBUG=true` en el archivo `.env`.
- **Permisos de archivos y cacheo:** Asegúrate de que las carpetas de almacenamiento y caché tengan los permisos necesarios para permitir la escritura. Esto es importante para evitar problemas en producción.

Además, se recomienda configurar un sistema de control de versiones como Git para mantener un seguimiento de los cambios en el código y facilitar la colaboración en equipo. El uso de ramas (branches) en Git permite desarrollar nuevas funcionalidades de forma aislada, lo cual evita conflictos y asegura un flujo de trabajo ordenado.

TAREA N° 07

Desarrollar una aplicación web en PHP con Framework.

1. SINTAXIS BÁSICA EN UN FRAMEWORK PHP.

- Estructura y convenciones del Framework.

La mayoría de los frameworks PHP modernos, como Laravel, Symfony y CodeIgniter, imponen una estructura de carpetas y convenciones que deben seguirse para facilitar el desarrollo y mantenimiento del código. La estructura generalmente incluye carpetas para Modelos, Vistas y Controladores (MVC), además de carpetas para configuraciones, rutas, recursos y migraciones de bases de datos.



[Logo de Symfony.](#)

Ejemplo en Laravel:

- **app/** – Contiene los modelos, controladores y otros componentes del núcleo de la aplicación.
- **resources/** – Incluye las vistas y archivos de lenguaje.
- **routes/** – Define las rutas de la aplicación en archivos como web.php y api.php.
- **config/** – Archivos de configuración de la aplicación, como database.php para la conexión a la base de datos.

Además de la estructura básica, algunos frameworks incluyen carpetas para migraciones, semillas de datos y test unitarios.

Estas herramientas permiten desarrollar y mantener un flujo de trabajo automatizado, asegurando que los cambios en la base de datos sean coherentes y probados a lo largo del proyecto.

Ejemplo en Laravel:

- **database/migrations/** – Contiene scripts que permiten versionar y sincronizar cambios en la base de datos.
- **database/seeds/** – Permite llenar la base de datos con datos de prueba para desarrollo y testing.
- **tests/** – Contiene pruebas unitarias y funcionales para verificar el funcionamiento del código en el framework.

- **Diferencias entre sintaxis del Framework y PHP nativo.**

Los frameworks PHP ofrecen sintaxis simplificada para muchas operaciones, ayudando a escribir menos código y a mejorar la legibilidad:

- **Controladores:** En un framework, los controladores son clases que manejan las solicitudes y retornan vistas o datos JSON, a diferencia de PHP nativo, donde se maneja toda la lógica en el archivo.
- **ORM (Object-Relational Mapping):** En lugar de consultas SQL en PHP nativo, los frameworks como Laravel utilizan ORM como Eloquent, que permite interactuar con la base de datos mediante métodos de objetos.

Ejemplo:

En PHP nativo, realizar una consulta es algo como:

```
$result = mysqli_query($conn, "SELECT * FROM users WHERE id = 1");
```

En Laravel:

```
$user = User::find(1);
```

2. FUNCIONES PRINCIPALES EN UN FRAMEWORK PHP.

- **Funcionalidades clave para desarrollo rápido.**

Los frameworks PHP incluyen funciones y herramientas que permiten desarrollar aplicaciones de forma rápida y organizada:

- **Autenticación y autorización:** Laravel, por ejemplo, tiene comandos de generación de autenticación (`php artisan make:auth`) que proporcionan una estructura de autenticación básica lista para usar.

Los frameworks también ofrecen CLI (Interfaz de Línea de Comandos), permitiendo ejecutar comandos para generar controladores, modelos y migraciones. Estas herramientas aceleran el desarrollo, ya que automatizan tareas repetitivas y estructuran los archivos con coherencia.

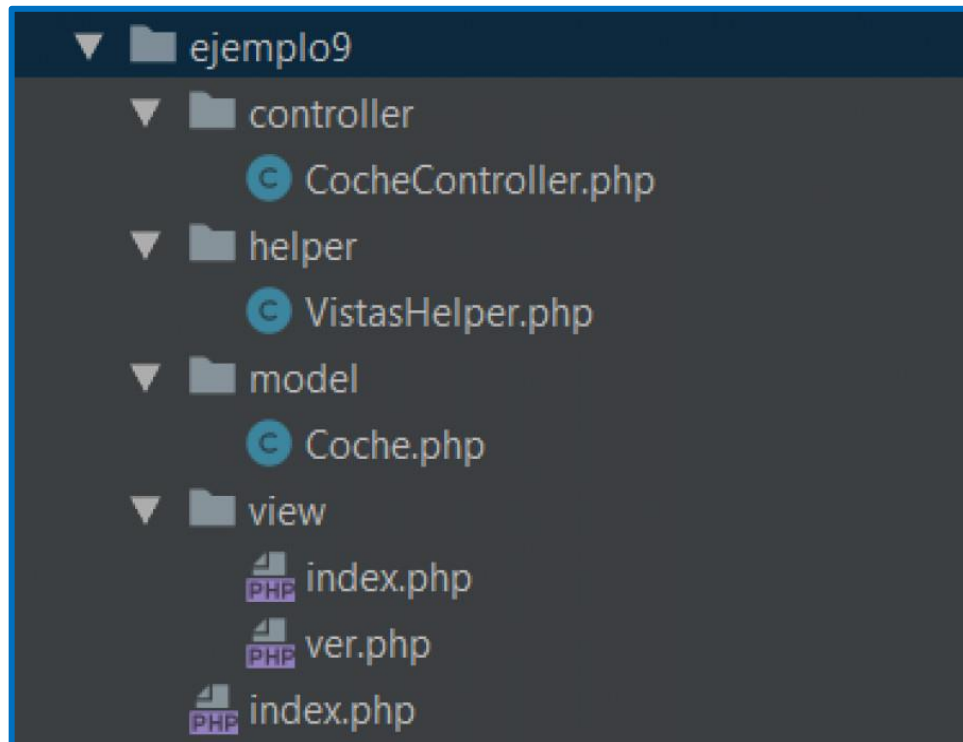
Ejemplo en Laravel:

```
php artisan make:model Product -m
```

Este comando genera un modelo Product junto con una migración para la base de datos.

- **Migraciones y Seeders:** Permiten versionar cambios en la base de datos y crear datos de prueba.
- **Rutas y Middleware:** Configurar rutas y aplicar middleware para controlar el acceso y modificar solicitudes de manera centralizada.
- **Uso de Helpers y funciones incorporadas.**

Los frameworks vienen con helpers (ayudantes) que permiten realizar operaciones comunes de manera rápida. Estos pueden incluir funciones para manipular arrays, cadenas, URLs y vistas, entre otros.



[Referencia de uso de helpers.](#)

Ejemplos de helpers comunes:

- **url():** Genera URLs completas.

- **view()**: Retorna una vista sin necesidad de escribir toda la ruta.
- **array_pluck()**: Extrae valores de un array multidimensional.

Además de los helpers básicos, algunos frameworks permiten definir helpers personalizados. Estos ayudan a mantener el código organizado, permitiendo crear funciones reutilizables que se pueden utilizar en todo el proyecto. Los helpers personalizados son útiles para encapsular lógica común que no está cubierta por los helpers nativos del framework.

3. CLASES PRINCIPALES DE UN FRAMEWORK PHP.

- **Modelos, vistas y controladores.**

La arquitectura MVC (Modelo-Vista-Controlador) es fundamental en los frameworks PHP, donde cada componente tiene su función:

- **Modelos:** Representan la estructura de datos y las interacciones con la base de datos.
- **Vistas:** Definen la interfaz de usuario y contienen el HTML que se envía al navegador.
- **Controladores:** Manejan la lógica de las solicitudes y deciden qué modelos utilizar y qué vistas mostrar.

Ejemplo Laravel:

```
// Controlador que utiliza un modelo para obtener datos y mostrarlos en una vista.
class UserController extends Controller {
    public function show($id) {
        $user = User::find($id);
        return view('user.profile', ['user' => $user]);
    }
}
```

Muchos frameworks también ofrecen controladores de recursos y controladores API. Estos controladores están especialmente diseñados para manejar solicitudes de tipo RESTful y simplificar la construcción de aplicaciones que requieren API o funcionalidades CRUD.

Ejemplo en Laravel:

```
php artisan make:controller ProductController --resource
```

Este comando genera un controlador con métodos predefinidos para realizar operaciones CRUD.

- **Clases auxiliares y componentes.**

Los frameworks incluyen clases auxiliares y componentes como servicios para envío de correos, manejo de sesiones, colas de trabajo y autenticación. En Laravel, por ejemplo, existen Facades que permiten acceder a componentes específicos como Mail, Auth y Cache de forma simplificada.

Ejemplo de Facades en Laravel:

```
// Enviar un correo electrónico usando la Facade Mail
Mail::to($user->email)->send(new WelcomeEmail($user));
```

Además, los frameworks suelen incluir servicios adicionales como colas para el procesamiento en segundo plano, eventos y listeners para gestionar flujos de eventos, y proveedores de servicios que permiten cargar módulos de la aplicación. Estas herramientas ayudan a gestionar tareas complejas y distribuidas dentro de una arquitectura sólida y escalable.

4. INTEGRACIÓN DE LIBRERÍAS EXTERNAS EN EL DESARROLLO CON FRAMEWORK.

- **Instalación y configuración de librerías con Composer.**

Composer es el gestor de dependencias estándar en PHP que permite añadir y gestionar librerías externas. Para instalar una librería, solo necesitas usar el comando `composer require nombre_libreria`. Composer se encarga de descargar e instalar la librería en la carpeta `vendor/`, y actualizar el archivo `composer.json` con la dependencia añadida.

Ejemplo:

```
composer require guzzlehttp/guzzle
```

Además de la instalación básica, Composer permite establecer versiones específicas y restricciones de versiones para asegurar la compatibilidad entre dependencias. También es posible crear scripts de Composer que ejecutan comandos específicos después de instalar o actualizar librerías.

Ejemplo de script en `composer.json`:

```
"scripts": {
    "post-update-cmd": [
        "php artisan optimize"
    ]
}
```

- **Interacción y compatibilidad con librerías externas.**

La integración de librerías externas en un framework PHP requiere verificar la compatibilidad y, en algunos casos, realizar configuraciones iniciales. Para utilizar una librería, se recomienda seguir estos pasos:

- **Comprobar la compatibilidad:** Verificar que la versión de PHP y el framework son compatibles con la librería.
- **Configuración inicial:** Muchas librerías requieren una configuración de parámetros, que generalmente se realiza en archivos de configuración del framework.
- **Importar y usar la librería:** En Laravel, por ejemplo, es común registrar servicios en el contenedor de servicios (app.php) o utilizar Facades para un acceso más directo.

Ejemplo de uso:

Para usar Guzzle, una librería de solicitudes HTTP, en Laravel:

```
use GuzzleHttp\Client;  
$client = new Client();  
$response = $client->get('https://api.example.com/data');
```

Es recomendable revisar la documentación y comunidad de cada librería antes de integrarla, ya que las actualizaciones pueden cambiar la compatibilidad con el framework. Asimismo, realizar tests de integración para verificar que las librerías funcionan correctamente en el contexto del proyecto es una buena práctica, asegurando que no afecten otras funcionalidades.

REFERENCIAS

- AJAX. (s/f). MDN Web Docs. https://developer.mozilla.org/es/docs/Learn/JavaScript/Client-side_web_APIs/Fetching_data
- ¿Cómo usar un servidor SMTP para enviar email y qué es? (2024). Mailjet. <https://www.mailjet.com/es/blog/emailing/servidor-smtp/>
- Completo, V. mi P. (s/f). *PHP OO: Primer proyecto con eclipse - métodos*. PHP MySQL. <http://mysqliphp.blogspot.com/2016/12/proyecto-eclipse-metodos-poo.html>
- dCreations, & Desarrollador De Software Freelance. (2023, septiembre 26). *Patrón MVC en PHP*. dCreations | Desarrollador De Software Freelance. <https://dcreations.es/cursos/curso-patrones-de-disenio-en-php-gratis/patron-mvc-en-php>
- Gomez, E. E. P. (2023, abril 21). *4 patrones de diseño que deberías saber para desarrollo web: Observador, Singleton, estrategia, y decorador*. freecodecamp.org. <https://www.freecodecamp.org/espanol/news/4-patrones-de-diseno-que-deberias-saber-para-desarrollo-web-observador-singleton-estrategia-y-decorador/>
- Gustavo, B. (2019, enero 18). *Los 8 mejores frameworks PHP para desarrolladores web*. Tutoriales Hostinger. <https://www.hostinger.es/tutoriales/mejores-frameworks-php>
- Krall, C. (s/f). *¿Qué es y para qué sirve Ajax? Ventajas e inconvenientes. JavaScript asíncrono, XML y JSON*. (CU01193E). Aprenderaprogramar.com. https://www.aprenderaprogramar.com/index.php?option=com_content&view=article&id=882:ique-es-y-para-que-sirve-ajax-ventajas-e-inconvenientes-javascript-asincrono-xml-y-json-cu01193e&catid=78&Itemid=206
- mail. (s/f). Php.net. <https://www.php.net/manual/es/function.mail.php>
- *Paradigma de la programación orientada a objetos*. (s/f). Github.io. https://ferestrepoca.github.io/paradigmas-de-programacion/poo/poo_teorias/concepts.html
- *WebSphere Application Server traditional 9.0.5.x*. (2024, junio 18). Ibm.com. <https://www.ibm.com/docs/es/was/9.0.5?topic=services-web>



RDA
RECURSO DIDÁCTICO PARA EL APRENDIZAJE